

Binary Information

Binary Name => PieIsMyFav

Language => C/C++

Arch => x86x64

Platform => Unix/Linux

```
$ file simple
```

simple: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0,

BuildID[sha1]=dae0509e4edb79719a65af37962b74e4cf2a8c2e, not stripped

Analysis

Static Analysis

```
gef> info functions
```

All defined functions:

Non-debugging symbols:

```
0x0000000000001000 _init
0x0000000000001030 puts@plt
0x0000000000001040 printf@plt
0x0000000000001050 __isoc99_scanf@plt
0x0000000000001060 __cxa_finalize@plt
0x0000000000001070 _start
0x00000000000010a0 deregister_tm_clones
0x00000000000010d0 register_tm_clones
0x0000000000001110 __do_global_ctors_aux
0x0000000000001150 frame_dummy
0x0000000000001155 main
0x00000000000011f0 __libc_csu_init
0x0000000000001260 __libc_csu_fini
0x0000000000001264 _fini
```

```
gef> disass main
```

Dump of assembler code for function main:

```
0x0000000000001155 <+0>: push rbp
0x0000000000001156 <+1>: mov rbp,rsp
0x0000000000001159 <+4>: sub rsp,0x10
0x000000000000115d <+8>: mov DWORD PTR [rbp-0x4],0x64
0x0000000000001164 <+15>: mov DWORD PTR [rbp-0x8],0xe
0x000000000000116b <+22>: mov DWORD PTR [rbp-0x10],0x0
0x0000000000001172 <+29>: mov DWORD PTR [rbp-0xc],0x0
0x0000000000001179 <+36>: lea rdi,[rip+0xe88] # 0x2008
0x0000000000001180 <+43>: call 0x1030 <puts@plt>
```

```

0x00000000000001185 <+48>: lea rdi,[rip+0xea8] # 0x2034
0x0000000000000118c <+55>: mov eax,0x0
0x00000000000001191 <+60>: call 0x1040 <printf@plt>
0x00000000000001196 <+65>: lea rax,[rbp-0x10]
0x0000000000000119a <+69>: mov rsi,rax
0x0000000000000119d <+72>: lea rdi,[rip+0xea7] # 0x204b
0x000000000000011a4 <+79>: mov eax,0x0
0x000000000000011a9 <+84>: call 0x1050 <__isoc99_scanf@plt>
0x000000000000011ae <+89>: mov edx,DWORD PTR [rbp-0x4]
0x000000000000011b1 <+92>: mov eax,edx
0x000000000000011b3 <+94>: add eax,eax
0x000000000000011b5 <+96>: add edx,eax
0x000000000000011b7 <+98>: mov eax,DWORD PTR [rbp-0x8]
0x000000000000011ba <+101>: add eax,edx
0x000000000000011bc <+103>: cdq
0x000000000000011bd <+104>: idiv DWORD PTR [rbp-0x4]
0x000000000000011c0 <+107>: mov DWORD PTR [rbp-0xc],eax
0x000000000000011c3 <+110>: mov eax,DWORD PTR [rbp-0x10]
0x000000000000011c6 <+113>: cmp DWORD PTR [rbp-0xc],eax
0x000000000000011c9 <+116>: jne 0x11d9 <main+132>
0x000000000000011cb <+118>: lea rdi,[rip+0xe7c] # 0x204e
0x000000000000011d2 <+125>: call 0x1030 <puts@plt>
0x000000000000011d7 <+130>: jmp 0x11e5 <main+144>
0x000000000000011d9 <+132>: lea rdi,[rip+0xe82] # 0x2062
0x000000000000011e0 <+139>: call 0x1030 <puts@plt>
0x000000000000011e5 <+144>: mov eax,0x0
0x000000000000011ea <+149>: leave
0x000000000000011eb <+150>: ret

```

End of assembler dump.

gef> x/s 0x2034

0x2034: "Qual o numero magico? "

gef> x/s 0x2008

0x2008: "Welcome to the wonderful world of assembly!"

gef> x/s 0x204b

0x204b: "%d"

gef> x/s 0x204e

0x204e: "Essa eh a sua flag!"

gef> x/s 0x2062

0x2062: "Try harder..."

gef>

Above is the disassembly of the **main()** function.

Program Workflow

- The binary takes an integer input which can be inferred because x/s 0x204b gives us **%d**
- If we give wrong integer then we get **Try harder...**
- If we give the right integer then we get **Essa eh a sua flag!**

Dynamic Analysis

Below is an image to demonstrate the logic behind the mathematical calculation of the program.

[illegible]

- You can set the breakpoint anywhere but set it somewhere above the instructions like I have done so that you can understand the calculation.
- I ran the program multiple times and I checked at the **idiv** instruction the division is always happening between **314(numerator)** and **100(denominator)**. The numerator and denominator were in **hex** formats so you can change it to decimal format and verify it yourself.
- Now the result when **314 divided by 100 is 3** and the **3** is compared with our user input value at the comparison. If you look at the image properly the input I gave was **30** which is later moved to **eax** register before the comparison happens and then later compared with the division result i.e **3**.
- **Hence, I reached my conclusion that the user input should be 3.**

Testing our input

```
$ ./simple
```

Welcome to the wonderful world of assembly!

Qual o numero magico? 3

Essa eh a sua flag!